

```
=====
Python OOPS
=====
```

```
** OOPS = Object Oriented Programming System

** OOPS is a programming approach where we design software using classes and objects.

** In real life, everything can be treated as an object.
```

```
Student -----> name, email, phno, course, fee
```

```
Employee -----> id, name, salary, project
```

```
Bank Account ----> account number, balance, branch name, deposit, withdraw
```

```
** Without OOPS, we usually write multiple variables and functions separately like below...
```

```
-----
```

```
student_name = "Ashok"
student_email = "ashok@gmail.com"
student_course = "Python"
```

```
def display_student():
    print(student_name)
    print(student_email)
    print(student_course)
```

```
display_student()
```

```
-----
```

```
Note: If we have 100 students, managing data becomes difficult.
```

```
** OOPS helps us to group related data and functions together.
```

```
=====
## Why OOP?
=====
```

```
** Code Re-Usability
```

```
** Code organization
```

```
** Security
```

```
** Easy Maintenance
```

```
** Reduce boiler plate code
```

```
** Building large scale applications
```

```
=====
## Main Concepts of OOP
=====
```

- 1) Class
- 2) Object
- 3) Constructor
- 4) Instance Variable
- 5) Instance Method
- 6) Encapsulation
- 7) Inheritance
- 8) Polymorphism
- 9) Abstraction

```
=====
## What is a Class?
=====
```

```
** Class is a blue print or template
```

```
** Class is a plan or model to define properties and behaviours
```

```
    -- properties nothing but variables to store the data
```

```
    -- behaviours nothing but methods to perform the action
```

Ex-1 :

```
    Student is a class
```

```
    student properties :  name, email, phno, course
```

```
    student behaviours : study( ), attend_class( ), write_exam ( )
```

Ex-2 :

```
    Employee is a class
```

```
    employee properties : id, name, salary
```

```
    employee behaviours : work(), attend_meetings(), take_leave(), take_salary()
```

Class syntax :

```
    class ClassName:
        statements
```

Class Example ::

```
    class Student:
        pass
```

Class Example ::

```
class Student:
    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)
```

```
=====
What is an Object?
=====
```

**** An object is a real instance created from a class.**

Ex : s1 = Student ()

Note: here s1 is an object of Student class.

```
-----
class Student:

    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)

s1 = Student()
s1.display()
-----
```

```
=====
## What is self?
=====
```

**** self is a reference variable that refers to the current object.**

**** self is used to access variables and methods inside a class.**

**** Whenever we create an object and call a method, Python automatically passes that object to self.**

**** self is not a keyword, but it is a standard naming convention.**

```
-----

class Student:
    id = 100
    name = "Ashok"

    def display(self):
        print(self.name)
        print(self.id)

    def study(self):
        print(self.name, "Student studying...")

# Creating object
s1 = Student()
```

```
# calling functions using object
```

```
s1.display() ## display(s1)
s1.study() ## study(s1)
```

```
-----
```

Note : When we call `s1.display()` and `s1.study()` , Python internally passes `s1` as `self`.

```
=====
## Constructor
=====
```

**** Constructor is a special method in Python.**

**** Constructor is automatically called when object is created for class.**

**** Constructor is used to initialize object data.**

**** In Python, constructor name is `__init__()`.**

```
=====
## Constructor Syntax
=====
```

```
class ClassName:
    def __init__(self):
        statements
```

```
=====
## Constructor Example
=====
```

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def display(self):
        print(self.name, "--", self.course)
```

```
s1 = Student("Ashok", "Python")
s1.display()
```

```
s2 = Student("John", "JAVA")
s2.display()
```

```
s3 = Student("GITA", "AI")
s3.display()
```

```
=====
## Instance Variables
=====
```

**** Variables declared using `self` keyword are called as instance variables.**

**** Instance variables belongs to object.**

**** Each obj can have separate values.**

```
## Example ::
```

```
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course
```

```
s1 = Student("Ashok", "Python")
s2 = Student("John", "AI")
```

```
print(s1.name)
print(s2.name)
```

```
=====
## Instance Methods
=====
```

```
** Methods which use self are called instance methods.
```

```
** Instance methods work with object data.
```

```
## Example
```

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display_result(self):
        if self.marks > 35:
            print(self.name, "Passed")
        else:
            print(self.name, "Failed")
```

```
s1 = Student("John", 100)
s2 = Student("Michael", 25)
```

```
s1.display_result()
s2.display_result()
```

```
=====
## Class Variables
=====
```

```
** Class variables are declared inside class but outside methods.
```

```
** Class variables are shared by all objects.
```

```
## Example ::
```

```
class Student:

    inst_name = "Ashok IT"

    def __init__(self, name):
        self.name = name

    def display_result(self):
        print(self.name, "--", Student.inst_name)
```

```
s1 = Student("John")
```

```
s2 = Student("Michael")
```

```
s1.display_result()
s2.display_result()
```

```
=====  
## Difference Between Instance Variable and Class Variable  
=====
```

```
** Instance variables
```

```
-- created using self  
-- belongs to object  
-- each object can have different values
```

```
** Class variables
```

```
-- created inside class but outside method  
-- belongs to class  
-- shared by all objects
```

```
=====  
What is class level method or static method ?  
=====
```

=> The method which doesn't depend on any object data that is called as static method.

=> Static methods can be called by using class name.

Ex ::

```
class Student:
```

```
    institute_name = "Ashok IT"
```

```
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks
```

```
    # Instance method
```

```
    def display_result(self):  
        print("Student Name:", self.name)  
        print("Marks:", self.marks)
```

```
        if Student.is_pass(self.marks):  
            print("Result: Pass")  
        else:  
            print("Result: Fail")
```

```
    # Static method
```

```
    @staticmethod  
    def is_pass(marks):  
        return marks >= 35
```

```
s1 = Student("Ravi", 80)  
s2 = Student("Kiran", 25)
```

```
s1.display_result()  
print("-----")  
s2.display_result()
```

```
=====
instance methods vs static methods
=====
```

```
** Instance methods
```

```
-- belongs to object
-- works with object data
-- uses self
-- will be called using object reference
```

```
** Static methods
```

```
-- Belongs to class logically
-- Doesn't work with object data
-- Doesn't use self
-- uses @staticmethod (optional)
-- will be called using classname
```

```
-----Revision-----
```

- 1) Why OOPS
- 2) Class
- 3) Object
- 4) self
- 5) instance variables
- 6) instance methods
- 7) Constructor
- 8) Class Variables
- 9) Static Methods

```
-----
```

```
=====
## Encapsulation
=====
```

```
=> Encapsulation means wrapping data and methods into a single unit.
```

```
=> Class is an example of encapsulation.
```

```
=> Encapsulation is used to protect data from direct access.
```

```
=====
## Public Variables
=====
```

```
=> Public variables can be accessed anywhere.
```

```
class Student:
```

```
    def __init__(self, name):
```

```
        self.name = name

s1 = Student("Ashok") # Student(obj, name)
s1.name = "John"
print(s1.name)
```

```
=====
## Private Variables
=====
```

=> Private variables are declared using double underscore __.

=> Private variables cannot be accessed directly outside the class.

=> To use private variables we need to write getter & setter methods

=> Setter method is used to update private variable data.

=> Getter method is used to read private variable data.

Ex :

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def set_balance(self, balance):
        self.__balance = balance

account = BankAccount(200)
# print(account.__balance) # error
print(account.get_balance())
account.set_balance(100)
print(account.get_balance())
```

```
=====
## Inheritance
=====
```

Parents ---> properties (cash, house, land).

Child ---> i can access my parents properties directly.

=> Inheritance means acquiring properties and methods from one class to another class.

=> We can achieve code re-usability using inheritance.

=> In inheritance concept we will use Parent and Child classes concept.

=> Parent class is also called base class or super class.

=> Child class is also called derived class or sub class.

Example::

```
class Parent:
    def house(self):
```

```
        print("Parent House")

class Child(Parent):
    def bike(self):
        print("Child Bike")

c = Child()
c.bike()
c.house()

===== Inheritance Example =====

class ChatBot:

    def welcome_user(self):
        print("Welcome to GEN AI Chatbot..")

    def collect_question(self, question):
        print("User Question : ", question)

class CustomerSupportBot(ChatBot):
    def solve_customer_issue(self):
        print("Solving Customer issue using AI")

class CourseCounsellingBot(ChatBot):
    def sugget_course(self):
        print("Suggesting Best Course Using Ai")

bot1 = CustomerSupportBot()
bot1.welcome_user()
bot1.collect_question("Why my outlook is not working ?")
bot1.solve_customer_issue()

print("-----")

bot2 = CourseCounsellingBot()
bot2.welcome_user()
bot2.collect_question("Which course is best for me ?")
bot2.sugget_course()

=====
## Types of Inheritance
=====

1) Single Inheritance

2) Multiple Inheritance

3) Multi Level Inheritance

4) Hierarchical Inhertience

5) Hybrid Inhertience

=====
## Single Inheritance
=====
```

```
** One child class inherits from one parent class.
```

Example :

```
class Parent:
    def parent_method(self):
        print("Parent method")

class Child(Parent):
    def child_method(self):
        print("Child method")

obj = Child()

obj.parent_method()
obj.child_method()
```

```
=====
## Multiple Inheritance
=====
```

```
** One child class inherits from multiple parent classes.
```

```
** Python supports Multiple Inheritance by following MRO (method resolution order)
```

```
** Check for method in current class first, if available execute and no need to check in parents
```

```
** If method not available in current class, check for the method in first parent, if available
execute it (no need to check in second parent)
```

```
** If method not available in first parent then only it will check for the method in second
parent..
```

Example::

```
...
class Father:
    def father_property(self):
        print("Father property")
    def m1(self):
        print("Father m1 method")

class Mother:
    def mother_property(self):
        print("Mother property")
    def m1(self):
        print("Mother m1 method")

class Child(Father, Mother):
    def child_property(self):
        print("Child property")
    def m1(self):
        print("Child m1 method")

obj = Child()
obj.father_property()
obj.mother_property()
obj.child_property()
obj.m1()
```

```
=====
## Multilevel Inheritance
=====
```

Child ----> Parent ----> GrandParent

**** One class inherits from another child class.**

Example::

```
...
class GrandParent:
    def grandparent_method(self):
        print("Grand Parent method")

class Parent(GrandParent):
    def parent_method(self):
        print("Parent method")

class Child(Parent):
    def child_method(self):
        print("Child Method")

obj = Child()
obj.child_method()
obj.parent_method()
obj.grandparent_method()
...
```

```
=====
## Hierarchical Inheritance
=====
```

**** Multiple child classes inherit from same parent class.**

Example ::

```
...

class Parent:
    def parent_method(self):
        print("Parent method")

class Child1(Parent):
    def child1_method(self):
        print("Child1 method")

class Child2(Parent):
    def child2_method(self):
        print("Child2 method")

c1 = Child1()
c2 = Child2()

c1.parent_method()
c1.child1_method()
print("-----")
c2.parent_method()
```

```
c2.child2_method()
```

```
...
```

```
=====
Hybrid Inheritance
=====
```

**** Hybrid inheritance means combining two or more inheritance types in a single program.**

Class A:

Class B:

Class C:

Class D:

B and C inherit from A (Hierarchical)

D inherits from both B and C (Multiple Inheritance)

Example :

```
...
```

```
class A:
    def method_a(self):
        print("Method from class A")
```

```
class B(A):
    def method_b(self):
        print("Method from class B")
```

```
class C(A):
    def method_c(self):
        print("Method from class C")
```

```
class D(B, C):
    def method_d(self):
        print("Method from class D")
```

```
obj = D()
obj.method_a()
obj.method_b()
obj.method_c()
obj.method_d()
```

```
...
```

```
=====
## Method Overriding
=====
```

**** If child class provides its own implementation for parent class method, it is called method overriding.**

Ex-1:

```
class Parent:
    def display(self):
        print("Parent display method")

class Child(Parent):
    def display(self):
        print("Child display method")

obj = Child()
obj.display()
```

Ex-2:

```
class AIModel:
    def generate_response(self, prompt):
        print("Generating response using general AI Model")

class OpenAIModel(AIModel):
    def generate_response(self, prompt):
        print("Generating response using OpenAI GPT Model")
        print("Prompt:", prompt)

class GeminiModel(AIModel):
    def generate_response(self, prompt):
        print("Generating response using Gemini Model")
        print("Prompt:", prompt)

model1 = OpenAIModel()
model1.generate_response("Explain Python Decorators")
print("=====")
model2 = GeminiModel()
model2.generate_response("Explain Python File handling")
```

```
=====
## super() Function
=====
```

**** super() is used to call parent class constructor or method from child class.**

Ex:-1 : Method

```
class Parent:
    def display(self):
        print("Parent display method")

class Child(Parent):
    def display(self):
        super().display()
        print("Child display method")

obj = Child()
obj.display()
```

Ex-2 :: Constructor

```
class Person:
    def __init__(self, name):
```

```
        self.name = name

class Student(Person):
    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def display(self):
        print(self.name)
        print(self.course)

s1 = Student("Ashok", "Python")
s1.display()
```

```
=====
## Polymorphism
=====
```

=> Polymorphism means one name with many forms.

=> Polymorphism means one method name behaves differently based on the object.

My Name is : Ashok

=> in class, i will act as a trainer

=> in ofc, i will act as employee

=> in inst, i will act as director(boss)

=> in home, i will act as family member

Ex-1 ::

```
class Dog:
    def sound(self):
        print("Dog Barks")
```

```
class Cat:
    def sound(self):
        print("Cat meows")
```

```
def animal_sound(i):
    i.sound()
```

```
dog = Dog()
cat = Cat()
```

```
animal_sound(dog)
animal_sound(cat)
```

Ex-2 ::

```
class UpiPayment:
    def pay(self, amount):
        print(f"Paid â,¹{amount} using UPI")
```

```

class CreditCardPayment:
    def pay(self, amount):
        print(f"Paid â,¹{amount} using Credit Card")

class DebitCardPayment:
    def pay(self, amount):
        print(f"Paid â,¹{amount} using Debit Card")

class CashPayment:
    def pay(self, amount):
        print(f"Paid â,¹{amount} using Hand Cash")

def make_payment(payment_method_type, amount):
    payment_method_type.pay(amount)

make_payment(CreditCardPayment(), 1000)
make_payment(DebitCardPayment(), 2000)
make_payment(CashPayment(), 5000)
make_payment(UpiPayment(), 10000)

```

```

=====
## Method Overloading
=====

```

** Writing same method multiple times with different parameters in same class is called as Method Overloading.

Ex-1 :

```

class Calculator:

    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c

cal = Calculator()
# print(cal.add(1,2))
print(cal.add(1,2,3))

```

Note: If we create methods with same name, latest method will replace previous method.

```

=====
## Abstraction
=====

```

** Abstraction means hiding implementation details and showing only required information.

Ex:

=> When we are driving a car, we use steering, brakes, accelerator
We do not need to know internal engine implementation.

```

=====
## Abstract Class
=====

```

** Abstract class is a class that contains abstract methods.

**** Abstract class cannot be used to create object directly.**

**** In Python, abstraction is implemented by using "abc" module.**

abc = Abstract Base Classes

Note: Abstract method is a method declared in parent class but implemented in child class.

abstract method : The method which doesn't contain body

Ex:

```
@abstractmethod
def pay(self, amount):
    pass
```

concrete method : The method which contains body

```
def payment_success_msg(self):
    print("Payment is successful, Thankyou...!!")
```

Note: When a class is extending properties from abstract class, that class must override all the abstract methods.

Ex

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):
```

```
    def __init__(self):
        print("ABC Constructor")
```

```
    @abstractmethod
    def pay(self, amount):
        pass
```

```
    def payment_success_msg(self):
        print("Payment is successful")
```

```
class UpiPayment(Payment):
```

```
    def pay(self, amount):
        print("Paid using UPI : ", amount)
        super().payment_success_msg()
```

```
class CreditCardPayment(Payment):
```

```
    def pay(self, amount):
        print("Paid using CreditCard : ", amount)
        super().payment_success_msg()
```

```
upi = UpiPayment()
upi.pay(120)
```

```
creditcard = CreditCardPayment()
creditcard.pay(12000)
```

Important Points

```
-- Abstract class can contain
```

```
    ** Abstract methods
```

```
    ** Concrete Methods
```

```
    ** Constructor
```

```
    ** Instance Variables
```

```
    ** Static Methods
```

Note: When we create object for Child class if child class doesn't have constructor then it will check parent class constructor.

```
## Example ::
```

```
from abc import ABC, abstractmethod
```

```
class Payment(ABC):
```

```
    def __init__(self):
        print("Payment Class Constructor")
```

```
    @abstractmethod
    def pay(self, amount):
        pass
```

```
    def payment_success_msg(self):
        print("Payment is successfull")
```

```
class UpiPayment(Payment):
```

```
    def pay(self, amount):
        print("Paid using UPI : ", amount)
        super().payment_success_msg()
```

```
class CreditCardPayment(Payment):
```

```
    def __init__(self):
        super().__init__()
```

```
    def pay(self, amount):
        print("Paid using CreditCard : ", amount)
        super().payment_success_msg()
```

```
upi = UpiPayment()
upi.pay(120)
```

```
credit = CreditCardPayment()
credit.pay(35000)
```

```
=====
## Important Points
=====
```

```
** OOP stands for Object Oriented Programming.
```

```
** Python supports Object Oriented Programming Principles.
```

- ** Class is a blue print / model / plan / template
- ** Class is used to define properties and behaviours
- ** Object is an instance of class
- ** Self refers to current object
- ** Constructor is used to initialize instance variables
- ** In python constructor name is `__init__()`
- ** Constructor is called automatically when object is created.
- ** Variables declared using self are instance variables.
- ** Every Object will maintain its copy of instance variables
- ** Class Variables are shared by all objects.
- ** Instance methods uses self as first parameter.
- ** Instance methods will be called by using obj reference
- ** Class methods will be called by using Classname.
- ** Encapsulation is used to protect data.
- ** Private variables are declared using double underscore (Ex: `__name`)
- ** We will write setter and getter methods for private variables.
- ** Inheritance improves code re-usability
- ** Child class can access Parent class members directly.
- ** Python follows MRO to identify which method to execute in inheritance flow.
- ** `super( )` is used to call parent class methods or constructor.
- ** Polymorphism means one name with many forms (behaviours)
- ** Writing same method in parent and child classes is called as Method Overriding.
- ** Method Overloading is not directly supported by Python.
- ** Abstraction is used to hide implementation details.
- ** Abstract class can be created using "abc" module in python.
- ** Abstract class can contain both abstract methods and concrete methods
- ** Python don't have interfaces concept.
- ** When a class extending Abstract class, then that class must implement all abstract methods.